

易可思科技 AI工程師

Coderbyte 測驗 — 完整教學講義

詳解版：原理說明、生活比喻、流程圖解、逐步範例，適合考前系統性理解

涵蓋：選擇/是非題 · 簡答/開放題 · 程式實作題 三種題型準備方向

每章附【原理】說明機制本身，並依 104

職缺敘述、操作說明手冊考古題、個人履歷技術棧交叉比對整理

考前快速複習請見另一份 `ecostek_exam_summary.pdf`（考試重點整理）



每個分支章節開頭都有【原理】段落，說明機制本身如何運作，而非只列重點條目

第八章（程式邏輯）額外附演算法評估總表+流程圖/表格圖解+各演算法出題機率評估

一 HTTP／Web 通訊基礎

從請求-回應循環，到 Redirect／REST／安全性的深入細節

【原理】HTTP 建立在 TCP 連線之上，本質是「無狀態」協定：每次 Request 都是獨立的，Server 預設不會記得你上一次做過什麼，這也是為什麼需要 Session/Token 額外模擬「狀態」的根本原因。Redirect 的運作原理是：Server 收到請求後判斷要轉址，回傳的 Response 不含真正內容，而是在狀態列放上 3xx 狀態碼、並在 Header 的 Location 欄位帶上新網址；瀏覽器的網路層解析到 3xx 狀態碼後，會「自動」對 Location 指定的網址發出全新的一次 Request——整個過程不需要 JavaScript 介入，是瀏覽器底層網路堆疊內建的行為，這與前端用 JS 做 window.location 轉址是不同機制。

HTTP 方法語意

- GET 查詢（可快取、無副作用）／POST 新增／PUT 整筆覆蓋／PATCH 局部更新／DELETE 刪除。PUT／GET／DELETE 具「冪等性 Idempotent」（同一請求重複呼叫，結果都相同），POST 通常不具。

狀態碼分類（精準記憶）

- 1xx 資訊／2xx 成功（200 OK）／3xx 轉址（301、302）／4xx 客戶端錯誤（400 格式錯、401 未驗證身份、403 已驗證但無權限、404 找不到）／5xx 伺服器錯誤（500）。

Redirect 精確流程（考古題）

- Server 回應 301（永久轉址）或 302（暫時轉址）狀態碼，並在 Response Header 的 Location 欄位放上新網址；瀏覽器解析到 3xx 狀態碼後，自動對 Location 指定的 URL 發出新的 Request。301 會被瀏覽器快取、下次直接跳新網址；302 每次都會重新問 Server。

RESTful API 設計原則

- 以「資源 Resource」為 URL 主體（用名詞而非動詞，如 /users/1）、用 HTTP 方法表達動作、Server 端無狀態（不記錄 Client 狀態，每個請求需自帶必要資訊）。

Session vs Token(JWT)

- Session：登入狀態存在 Server 端的記憶體或資料庫，Server 每次都要查表確認身份，較耗資源但可即時撤銷登入。Token/JWT（JSON Web Token）：把身份資訊加密簽章後整包交給 Client 自行保存，Server 端免查表、易水平擴展，但簽發後較難即時作廢。

生活比喻

HTTP 就像「寄信」：每封信（Request）都是獨立的，郵差（Server）處理完、回信（Response）後就不記得你是誰，除非信封上你自己夾了一張「會員卡」（Session/Token）提醒對方你是誰。Redirect 就像「查無此人、新地址在此」的退信條，郵差看到條子上的新地址，直接把信改送到新地址去，收件人完全不用自己重寫信封。

實務小例子（面試可直接這樣講）

面試被問 redirect 用途時可以說：「像使用者登入後，後端常會回傳 302 導向到首頁；我在搶票系統裡，也用類似邏輯讓未登入的使用者被導回登入頁，這樣前端不用自己寫判斷邏輯，後端一個 status code 就處理掉了。」

選擇/是非題	簡答/開放題	程式實作題
配對狀態碼與情境（如 401 vs 403 的差異：401＝沒登入、403＝登入了但沒權限）。	考古題原題「請簡短說明 HTTP redirect 是如何運作的」→ 直接背上方精確流程作答，並點出 status code 與 Location header 兩個關鍵字。	可能考「寫一個回傳 redirect 的 API」或「解析 URL 參數」，注意 FastAPI 用 RedirectResponse、Flask 用 redirect()。

二 作業系統與併發基礎

從 Process/Thread 定義，到 Race Condition／Deadlock／分散式鎖

【原理】單顆 CPU 核心同一時間只能執行一件事，作業系統靠「時間切片+Context Switch（切換執行狀態）」讓多個 Process/Thread 輪流獲得極短的執行時間，切換速度快到人類感覺不出來，才產生「同時執行」的錯覺，這就是 Concurrency 的底層原理。Race Condition 之所以發生，是因為像 `count += 1` 這種看似一行的操作，實際上是「讀取值→運算→寫回」三個獨立步驟，若兩個 Thread 在讀取之後、寫回之前被切換執行，就會用舊值覆蓋新值，導致一次更新憑空消失。Lock/Mutex 能解決這個問題，原理是利用 CPU 提供的「原子操作指令 Atomic Instruction」（如 Compare-And-Swap），確保「檢查鎖是否被佔用」與「佔用鎖」這兩件事不會被其他 Thread 插隊打斷。Redis 分散式鎖能跨機器生效，是因為 Redis 指令執行本身是單執行緒、依序處理的，多台伺服器同時送出 SET NX 時，Redis 會依序處理、只有第一個會成功，其餘全部失敗，等同把「搶鎖」這個判斷集中到單一節點原子化執行。

Process（行程）

- 作業系統配置資源的基本單位，每個 Process 擁有獨立的記憶體空間，Process 之間互不干擾，但溝通成本較高（需透過 IPC，行程間通訊）。

Thread（執行緒）

- Process 底下實際被 CPU 排程執行的單位，同一 Process 內的多個 Thread「共享同一塊記憶體空間」，因此溝通快，但也因此容易互相搶奪同一份資料。

一句話精確對比

- Process 是各自獨立的房子；Thread 是同一間房子裡的家人，共用同一個冰箱（記憶體），所以動作快，但也會搶東西吃（race condition）。

Concurrency vs Parallelism

- Concurrency（並發）：單核心透過時間切片，交錯執行多個任務，看起來像同時發生。Parallelism（平行）：多核心真正同時執行多個任務。

Race Condition（你的搶票案例）

- 多個 Thread/Process 同時讀寫同一份共享資料，且最終結果會因「執行順序」不同而不同，導致資料錯亂（如同一張票被兩人搶到）。

Critical Section 與 Lock/Mutex

- Critical Section（臨界區）是會存取共享資源、必須避免被多方同時進入的程式區塊；Lock/Mutex 是確保同一時間只有一方能進入該區塊的機制。

Deadlock（死鎖）四要件

- 互斥（資源不可共用）、佔有並等待、不可搶奪、循環等待——四個條件同時成立才會死鎖，破壞任一條件即可避免。

分散式鎖（Redis SET NX）

- 多台伺服器情境下，本機的 Lock 無法跨機器生效，因此用 Redis 的 SET key value NX（key 不存在時才設值成功）當作跨伺服器的搶鎖機制，搶到鎖的人才能繼續操作，並設定 TTL（存活時間）避免持有者當機造成永久卡死。

生活比喻

Process 像不同的「家庭」，各自有獨立的房子（記憶體）；Thread 像同一家庭裡的成員，共用同一個廚房和冰箱，動作互相配合快，但也可能同時去搶最後一顆雞蛋而打架（race condition）。Lock 就像「廚房裡的單人洗手間」，同一時間只能一個人進去用，門外的人要排隊等，確保不會兩人同時搶用馬桶。

實務小例子（面試可直接這樣講）

「我在搶票系統中，多人同時搶同一個座位就是典型 race condition，我用 Redis SET NX 當作『搶門把』的機制——誰先搶到門把（鎖），誰才能進去劃位，其他人要嘛失敗要嘛等待，確保不會兩個人同時劃到同一個位子，這也是我壓力測試 10~600 人模擬時要驗證的核心。」

選擇/是非題	簡答/開放題	程式實作題
是非題常考「Thread 共用記憶體、Process 不共用」這類敘述的正確性判斷。	考古題原題直接背上方 Process/Thread 定義＋一句話比喻，再補一句「因為我在搶票系統中用 Redis SET NX 處理過高併發座位衝突」加分。	可能考模擬鎖的邏輯或計數器安全性問題（如多執行緒下 count+=1 為何會出錯）。

三 Git 版本控制

從 Commit/Branch 基本模型，到 Merge/Rebase 差異

【原理】Git 底層是一個「內容定址 Content-Addressable」的資料庫：每一次 commit 都會把當下所有檔案的快照存成物件，並用內容算出的雜湊值（SHA hash）當作該物件的唯一身分證，commit 物件內還記錄了「上一個 commit 是誰」，所以整個歷史其實是一條由雜湊值互相指向串起來的鏈（有分支合併時則是圖）。Branch 之所以「建立成本極低」，是因為它並不複製任何檔案，只是新建一個存著某個 commit 雜湊值的小指標檔；HEAD 則是另一個指標，指向「你目前站在哪個 branch（或哪個 commit）上」，切換分支等於改變 HEAD 指向誰。Merge 的原理是三方合併演算法：比對「共同祖先 commit」與「兩條分支各自最新版本」的差異，自動合併沒有衝突的變更，衝突的部分才需要人工介入；Rebase 則是把你的每個 commit 依序「重新套用」到新的基底 commit 上，本質是逐一重放差異並產生全新的 commit 雜湊值，因此歷史看起來乾淨，但原本的 commit 已經被取代。

Commit

- 一次程式碼變更的「快照」，每個 commit 有唯一的 hash 值，記錄變更內容、作者與時間。

Branch（考古題核心）

- 「指向某個 commit 的可移動指標」，本身不複製程式碼；建立分支的成本極低，因為只是新增一個指標。

HEAD（考古題核心）

- 指向你目前所在的 branch（或直接指向某個 commit，稱為 detached HEAD）；換分支時 HEAD 就跟著改指向。

考古題敘述判斷

- 「A branch is a reference to a commit, and HEAD is a reference to the current commit or branch you have checked out.」— 這句話是正確敘述，回答 True。

Merge vs Rebase

- Merge：保留兩條分支各自的歷史，並新增一個合併 commit，安全但歷史較亂。Rebase：把你的 commit 依序「接」到目標分支最新的 commit 後面，歷史線性乾淨，但會改寫 commit hash，不建議對已推送到共用分支的紀錄做 rebase。

三態工作流程

- Working Directory（工作目錄，實際檔案）→ git add → Staging Area（暫存區）→ git commit → Repository（版本庫）。

Conflict 處理流程

- 同一段程式碼被兩邊修改時，Git 無法自動判斷取捨，需人工開啟衝突檔案，選擇/合併保留內容後，git add 標記解決，再完成 commit。

生活比喻

Git 的版本歷史像「遊戲存檔點」，每個 commit 是一次存檔，記錄當下進度的完整快照；Branch 像「平行世界的分支劇情線」，你可以從某個存檔點開一條新故事線去嘗試新玩法，不會影響原本主線；HEAD 就是「你目前正在玩哪一條故事線的哪一個存檔點」。GitHub 則像把這些存檔放到雲端，讓隊友能一起讀取、各自開分支線發展劇情，最後再把好的劇情線合併回主線。

實務小例子（面試可直接這樣講）

「在 MineMarket 專案裡，我們四人各自開 feature branch 開發不同模組（認證/商品/購物車/後台），開發完各自發 PR 合併回主分支，這樣主線隨時保持可運行狀態，不會因為某人還沒寫完的功能影響到其他人，衝突也能在 PR 階段就先處理掉，不會拖到最後才爆炸。」

選擇/是非題

簡答/開放題

程式實作題

是非/選擇題型態最常見，重點在「定義用詞精不精確」，尤其 branch＝指標、不是複製。	可能被問「你如何處理多人協作的 merge conflict」，可結合 MineMarket 四人協作專案經驗回答。	較少直接考 code，但可能考「說明你會用什麼指令解決某情境」。
--	--	----------------------------------

四 資料庫基礎（SQL／NoSQL／Redis）

從關聯式資料庫本質，到索引效能與快取策略

【原理】關聯式資料庫（SQL）的核心承諾是 ACID 交易保證，這是靠「先寫日誌再寫資料 Write-Ahead Logging」達成的：任何變更都先記錄到日誌檔，確認日誌寫入成功後才真正套用到資料本體，一旦中途當機，重開機時可依日誌回放或回滾，這是 Durability（持久性）與 Atomicity（原子性）的實作基礎；Isolation（隔離性）則透過鎖定機制或 MVCC（多版本並發控制，讓讀取者看到資料的某個歷史快照，不被其他人正在寫入的內容干擾）達成。索引（Index）的原理是額外建立一個「排序過的資料結構」（多為 B-Tree 平衡樹），讓查詢可以用類似二分搜尋的方式跳著找，把原本要逐筆掃描的 $O(n)$ 查詢降到 $O(\log n)$ ，代價是每次寫入/更新都要同步維護這個排序結構，因此寫入變慢。NoSQL（如 MongoDB）捨棄固定 Schema 與複雜 JOIN，換取「文件結構彈性」與「水平擴展容易」，本質是用一致性/複雜查詢能力去交換擴展性與開發彈性。Redis 之所以快，是因為它把資料整個放進記憶體（RAM）而非硬碟，讀寫速度比硬碟快上百倍，代價是容量受限於記憶體大小、且需額外機制（如 RDB/AOF）保證斷電不遺失資料。

SQL vs NoSQL 選型依據

- SQL（如 MySQL）：資料結構固定（Table/Schema）、天生支援複雜關聯查詢與交易一致性。NoSQL（如 MongoDB）：結構彈性（可存不固定欄位的 JSON 文件）、易水平擴展，適合資料格式高度不固定的情境（你 Threat Intel Agent 選 MongoDB 的理由）。

ACID（交易四要素，精準定義）

- Atomicity 原子性（要嘛全部成功要嘛全部失敗）／Consistency 一致性（交易前後資料維持正確狀態）／Isolation 隔離性（並行交易互不干擾）／Durability 持久性（一旦提交，資料永久保存不因當機遺失）。

Index（索引）

- 類似書本目錄，透過額外的 B-Tree 資料結構加速查詢；代價是「拖慢寫入速度」與「佔用額外儲存空間」，因此不是欄位越多加索引越好。

Normalization（正規化）

- 透過拆表消除資料重複與更新異常，但過度正規化會增加查詢時的 JOIN 成本，需視情境權衡（正規化 vs 查詢效能）。

Redis 常見資料結構與場景

- String（快取單一值/計數器）、Hash（存物件屬性）、List（訊息佇列）、Set（去重/標籤）、Sorted Set（排行榜/依分數排序）。

Redis TTL 與快取策略

- TTL（Time To Live，存活時間）設定資料自動過期時間，避免快取資料永久佔用記憶體或資料過期未更新；常見策略為 Cache-aside（查快取沒有才查資料庫並寫回快取）。

生活比喻

SQL 資料庫像「制式表格的檔案櫃」——每個抽屜（Table）都先定義好欄位格式，資料一定要照格式填；NoSQL 像「便利貼牆」，每張便利貼（文件）想寫什麼、寫多少欄位都可以，彈性但沒那麼規整。Index 就像書本的目錄頁，不用一頁一頁翻找內容，直接看目錄跳頁查找。Redis 快取就像「隨手可拿的桌面文件」而非「要跑一趟檔案室調閱的公文」，快很多，但桌面（記憶體）空間有限，放不下太多東西。

實務小例子（面試可直接這樣講）

「Threat Intel Agent 裡我選 MongoDB，是因為不同情資來源（WHOIS/AbuseIPDB/VirusTotal）回傳的欄位結構都不一樣，如果用 SQL 我得為每個來源另外設計/維護固定 Table 結構，MongoDB 讓我直接把整包 JSON 存進去，開發彈性高很多，這正是資料結構固定與否這個判準的實際應用。」

選擇/是非題	簡答/開放題	程式實作題
可能考 ACID 四要素配對、或 SQL/NoSQL 特性判斷題。	很適合結合專案回答：「為何 MineMarket 用 MySQL、Threat Intel Agent 用 MongoDB」——用「資料結構固定性」這個判準來回答最精準。	可能考基本 SQL 查詢語法（SELECT/JOIN/WHERE）或簡單的 Redis 指令操作。

五 後端框架與非同步機制

從同步/非同步本質，到 FastAPI 與 Flask 架構差異

【原理】非同步（async/await）的核心機制是「事件迴圈 Event Loop」：整個程式其實只有一條主要執行緒，當程式遇到 I/O 操作（如等資料庫回應、等外部 API 回傳）時，不會傻傻卡住等待，而是把這個「還沒完成的任務」登記起來、讓出控制權去處理其他已經就緒的任務；等作業系統回報 I/O 完成，Event Loop 才把控制權還給原本的任務繼續往下執行。這與傳統多執行緒的差異在於：非同步用「一條執行緒＋任務切換」就能同時應付大量等待中的連線，不需要為每個連線開一條 Thread，因此在「大量連線、但多數時間都在等 I/O」的場景（如高併發搶票 API）遠比同步多執行緒省資源，這正是 WSGI（同步標準介面，Flask 預設每個 worker 一次只能處理一個請求）與 ASGI（非同步標準介面，FastAPI 搭配 Uvicorn，一個 worker 能同時交錯處理大量連線）在高併發表現上差異懸殊的根本原因。

同步 vs 非同步（async/await）

- 同步：一個任務沒做完，下一個任務要排隊等待。非同步：遇到 I/O 時先讓出控制權去處理其他任務，等 I/O 完成再回來繼續，適合「I/O bound（等待輸入輸出為主）」而非「CPU bound（運算量為主）」的場景。

WSGI vs ASGI

- WSGI（Flask 預設）：同步標準介面，一次處理一個請求。ASGI（FastAPI 用 Uvicorn）：支援非同步與 WebSocket，能用少量資源處理大量併發連線，這正是你搶票系統選 FastAPI + Uvicorn 處理高併發搶票請求的理由。

FastAPI vs Flask 精準比較

- FastAPI：內建 Pydantic 資料驗證、自動產生 API 文件（Swagger）、原生支援 async，適合高併發 API。Flask：輕量、彈性高、生態成熟，適合搭配樣板引擎做伺服器端渲染（如你 MineMarket 用 Jinja2）。

Middleware / Decorator

- 在請求進入實際邏輯之前（或回應送出之前）統一處理的邏輯層，如身份驗證、Log 紀錄、例外處理；對應你在 MineMarket 專案中自製的路由守衛、參數解析、資料庫交易三類 Decorator。

ORM 概念

- 把資料庫的資料表對應成程式語言中的物件（Object-Relational Mapping），讓開發者用寫程式碼的方式操作資料庫，不必手刻 SQL 字串，但需理解其底層仍會轉譯成 SQL 執行。

生活比喻

同步處理像「一次只能接一組客人的櫃檯」，前面客人沒辦完事，後面全部要排隊等；非同步像「餐廳外場一人服務多桌」——幫 A 桌點完餐送進廚房後，不會站在廚房前乾等，而是先去幫 B 桌點餐，等 A 桌的餐好了廚房會通知服務生再送過去。FastAPI 就像訓練有素、能同時周旋多桌的外場；Flask 則像穩紮穩打、一次專心服務一組客人的傳統櫃檯，服務品質同樣好，只是同時能接的客人數量不同。

實務小例子（面試可直接這樣講）

「我的搶票系統選 FastAPI + Uvicorn，是因為搶票當下會有大量使用者同時打 API、且多數時間卡在等資料庫/Redis 回應，用非同步能讓少量伺服器資源同時應付大量等待中的請求，比傳統同步阻塞的做法撐得住更高流量；MineMarket 則因為是伺服器端渲染的電商網站、對併發要求沒那麼極端，選 Flask 搭配 Jinja2 開發效率更高。」

選擇/是非題	簡答/開放題	程式實作題
可能考「FastAPI 與 Flask 的差異」直接配對題。	重點題型，建議準備「為什麼搶票系統選 FastAPI、MineMarket 選 Flask」的一句話理由（併發需求 vs 伺服器端渲染需求）。	可能考寫一個簡單 endpoint 或 middleware/decorator 實作。

六 系統設計基礎（高併發／擴展性）

從快取與限流策略，到佇列與水平擴展

【原理】系統設計題的本質是「資源有限、流量無限」的取捨問題：Cache 的原理是把「昂貴但不常變動的查詢結果」暫存在快取取更快的地方（記憶體），下次同樣查詢直接命中快取、不必再打資料庫，用「額外的儲存空間」換「查詢速度」；但快取與資料庫是兩份獨立的資料，一旦沒設計好更新/失效機制，就會出現「快取穿透」（查詢的資料根本不存在，快取永遠不會命中，每次都直接打穿到資料庫）、「快取擊穿」（單一極熱門的 key 剛好過期，瞬間大量請求同時打到資料庫）、「快取雪崩」（大量 key 在同一時間點一起過期，資料庫瞬間被排山倒海的請求壓垮）三種典型陷阱。限流（Rate Limiting）的原理則是主動「犧牲部分請求」換取系統整體不被壓垮：Token Bucket（令牌桶）演算法用固定速率持續產生令牌，請求要先拿到令牌才能被處理，令牌用完的請求就被拒絕或排隊，藉此把瞬間尖峰流量整形成系統可負荷的平穩速率。水平擴展能生效的前提是「無狀態設計」——如果 Session 狀態存在單一台主機的記憶體裡，流量被負載平衡器分散到別台機器時就會找不到狀態，所以水平擴展通常需要搭配把狀態移到共用的外部儲存（如 Redis）。

Cache 策略與陷阱

- Cache-aside：先查快取，沒有才查資料庫並寫回快取。三大陷阱——快取穿透（查詢根本不存在的資料，每次都打到資料庫）、快取擊穿（單一熱門 key 過期瞬間大量請求打到資料庫）、快取雪崩（大量 key 同時過期，瞬間壓垮資料庫）。

Rate Limiting（限流）

- 限制單位時間內的請求數量，避免系統被瞬間流量壓垮；常見概念為 Token Bucket（固定速率產生令牌，請求需消耗令牌才能執行）。

水平擴展 vs 垂直擴展

- 垂直擴展：升級單台主機規格（有上限、單點風險）。水平擴展：增加主機數量分攤流量（需搭配負載平衡與無狀態設計）。

Message Queue 用途

- 任務丟進佇列由後端非同步消化，達到「解耦」（發送方不需等待處理完成）與「削峰」（把瞬間尖峰流量攤平處理）的效果，例如訂單成立後的道具發放可排隊補發（對應你 MineMarket 玩家離線補發道具的設計）。

實戰對應（搶票系統）

- 高併發座位鎖用 Redis SET NX 分散式鎖；壓力測試 10~600 人模擬找出多程序下才出現的 race condition，完整走過根因分析→修補→回歸驗證。

生活比喻

Cache 像「櫃檯前的常用零錢盒」，常用的零錢不用每次都跑去金庫（資料庫）拿，直接從零錢盒（快取）拿更快；但零錢盒要定期補貨/核對，不然可能給錯錢（快取與資料庫不一致）。限流（Rate Limiting）像「遊樂設施門口的排隊閘門」，用發放通行證的速度控制入場人潮，避免園區裡面被瞬間擠爆。

實務小例子（面試可直接這樣講）

「我在搶票系統做壓力測試時，模擬 10~600 人同時搶票，發現多程序併發下才會出現的 race condition，這讓我理解「系統設計」不能只憑感覺，一定要實際壓測驗證極限在哪裡，這也是我履歷特別強調「完整走過根因分析→修補→回歸驗證」的原因。」

選擇/是非題	簡答/開放題	程式實作題
較少出現，但可能考快取策略相關的觀念判斷。	最可能考「請設計一個能處理高流量的系統」這類申論題，直接套用你搶票系統的 Redis 鎖 + 壓力測試 + race condition 修補流程回答，最有說服力。	較少直接考code，但若出現，多半是限流或去重邏輯的簡化版本。

七 AI/LLM/Agent 應用

職缺核心考點：從 Prompt 基礎，到 Agent 自主決策架構

【原理】LLM（大型語言模型）的生成機制是「自回歸 Autoregressive」：模型每次只根據「已經產生的所有文字（含使用者輸入）」，去預測「下一個最可能出現的 Token（詞元，文字被切分後的最小處理單位）」，一個一個 Token 接龍式地生成，而不是一次想好整段話。模型能理解上下文，靠的是 Transformer 架構中的「自注意力機制 Self-Attention」——在處理某個 Token 時，模型會同時計算它與句子中「所有其他 Token」的關聯權重，藉此決定該把注意力放在哪些字詞上，這也是為什麼 LLM 能處理長距離語意關聯（例如句首的主詞和句尾的動詞對應）。Tool Use（工具呼叫）的原理是：在系統提示詞中把每個可用工具的名稱、用途、參數格式描述給模型，模型在生成回應時，若判斷當下情境需要外部資訊，會輸出一個「結構化的工具呼叫請求」而非純文字，應用程式攔截這個請求、實際執行對應的函式，再把執行結果重新包裝成一則訊息餵回給模型，模型才能根據新資訊繼續推理——這一來一回的循環就是 Agent 能「自主查資料」的底層機制。

Token 與 Context Window

- LLM 把文字切成 Token 作為處理單位；Context Window（上下文窗口）是模型一次能「看到」的 Token 上限，超過上限的資訊模型看不到，需靠設計（如摘要、檢索）解決。

Prompt Engineering 基本技巧

- 給明確指令與輸出格式要求、提供範例（few-shot，少量示例學習）、給予角色設定與情境限制，目的是降低模型輸出的不確定性。

Tool Use / Function Calling

- 讓 LLM 不只是純聊天，而是可以「決定呼叫哪個外部工具/API」取得即時資料或執行動作，再把工具回傳的結果納入後續推理。

Agent 與傳統 Rule-based 系統的差異

- Rule-based（規則式）：開發者事先寫死「什麼情況做什麼事」。Agent：由 LLM 自主判斷「當下該查哪些資料、查幾次、何時已經足夠可以下結論」，決策過程本身具備彈性與不確定性。

強制結構化輸出（你的專案做法）

- 用 tool_choice 強制模型第二輪一定呼叫指定的「回報函式」，確保輸出一定是可被程式解析的結構化 JSON，取代早期單靠 prompt 要求格式、但仍頻繁解析失敗的做法——這是「讓 AI 輸出穩定可用」的關鍵工程手法。

速率限制與快取因應

- 外部 API（如 VirusTotal 每分鐘 4 次、AbuseIPDB 每日 1000 次）多半有呼叫次數限制，需搭配快取（如 24 小時 TTL）降低重複呼叫，避免超過額度。

決策可追溯性

- 完整記錄 Agent 每一步查了什麼、耗時多久、是否命中快取，讓「判斷」過程可被事後稽核，而非只回傳一個無法解釋的最終結論。

Prompt Engineering 技巧詳解（實務常用手法）

【原理】Prompt Engineering 之所以有效，是因為 LLM 的輸出本質是「根據輸入內容去預測最可能接續的內容」，輸入（Prompt）給的資訊越明確、結構越清楚，模型可預測的「合理接續」範圍就越窄，輸出自然越穩定可控；反之模糊的指令會讓模型有大量同樣「合理」的接續方式可選，輸出容易飄忽不定。以下技巧都是圍繞這個原理設計。

Zero-shot（零範例）

- 不給任何範例，直接下指令，仰賴模型本身既有能力回答；適合簡單、模型本來就熟悉的任務（如翻譯、簡單分類）。

Few-shot（少量範例學習）

- 在 Prompt 中提供幾組「輸入→輸出」的範例，讓模型模仿範例的格式與邏輯再處理新的輸入；當你需要特定輸出格式或風格時效果顯著優於 zero-shot。

Chain-of-Thought CoT（思維鏈）

- 要求模型「先一步步想清楚邏輯，再給出最終答案」，而非直接跳答案；讓推理過程顯式化，能大幅提升複雜推理/數學/多步驟問題的準確度，代價是回應變長、耗費更多 Token。

角色設定 Role Prompting

- 在 Prompt 開頭給模型一個身分與情境（如「你是一位資安分析師，請根據以下情資判斷風險等級」），引導回答的語氣、知識取向與判斷標準；你的 Threat Intel Agent 就是用這個手法讓模型聚焦在資安判斷情境。

明確輸出格式限制

- 直接指定輸出必須符合的結構（如「只能回傳 JSON，欄位為 risk_level 與 reason」），減少後續程式解析失敗的機率；更嚴謹的作法是搭配 API 提供的 tool_choice／JSON mode 強制輸出符合 schema，而非只靠文字描述期望格式（你的專案採用的正是這種更嚴謹的作法）。

任務拆解 Task Decomposition

- 把一個大而複雜的任務拆成多個小步驟，分次或分階段下指令，而不是一次把所有要求塞進同一個 Prompt；你的 Agent 兩輪 Tool Use 架構（先查資料、再強制輸出報告）就是任務拆解的實例。

提供限制／負面指引

- 明確告訴模型「不要做什麼」（如「不要輸出任何除了 JSON 以外的文字」「不要臆測沒有依據的資訊」），能有效減少模型輸出多餘內容或編造答案（Hallucination，幻覺）的機率。

迭代優化 Prompt Iteration

- Prompt 很少一次就寫到最佳版本，實務作法是先寫一版、觀察實際輸出結果、找出不符預期之處，再針對性調整措辭或補充範例，反覆幾輪收斂到穩定輸出，是日常開發中持續在做的事。

Temperature 溫度參數

- 控制模型生成時的隨機性：數值越低（接近 0）輸出越穩定保守、適合需要一致性的任務（如結構化資料抽取）；數值越高輸出越多樣有創意，但也越不穩定，適合需要發想的任務（如文案發想）。

生活比喻 LLM 生成文字像「超強接龍遊戲」，每次只看前面所有字，猜下一個最合理的字接上去，一個字一個字生成，不是一次把整段話都想好。Tool Use 就像「總機客服」：當你問的問題超出總機自己能回答的範圍，總機會幫你轉接給對的專責窗口（外部工具/API），等窗口回覆後，再由總機統整答案告訴你，而不是總機自己憑印象亂猜。 實務小例子（面試可直接這樣講） 「我的 Threat Intel Agent 就是這種『總機』角色：使用者輸入一個 IP，Agent（總機）判斷該去問 WHOIS 還是 VirusTotal 這些『專責窗口』，拿到回覆後再統整成一份結構化風險報告，而不是憑空瞎猜答案，這也是我在意的『可解釋性』所在。」		
選擇/是非題	簡答/開放題	程式實作題
可能考 Prompt/Token/Tool Use 名詞定義配對。	本章是申論題重點戰場，直接用你的 Threat Intel Agent 案例作答：兩輪 Tool Use 架構、強制結構化輸出、決策軌跡記錄、風險判斷與執行權責分離。	較少直接考 code，但若考，可能是設計一個簡化版 tool-calling 流程或 prompt 撰寫。

八 程式邏輯與演算法基礎

對應 Coding Challenge (Math Challenge)，純手寫不能查 AI

【原理】操作手冊列出的 2 題 Coding Challenge 皆為「Math Challenge」，且系統會偵測切換分頁/複製貼上，這代表這兩題必須靠自己當場手寫。選演算法的核心原理是「用額外的資訊（排序、雜湊、記憶）換取時間」：Big O（時間複雜度）衡量的是「當資料量 n 越來越大時，執行時間成長的速度」而非實際跑幾秒；下筆前先估計 n 的量級，若題目資料量小，暴力解（Brute Force，窮舉所有可能）通常最快寫完也最不易出錯，只有在資料量大、暴力解會超時的情況下，才需要換更聰明（但通常也更複雜）的演算法。以下先給「本次最可能出現」的基礎數學邏輯題，接著給一份「業界常用演算法總表」涵蓋你在校所學的進階題型，並逐一評估在這次測驗出現的機率。

時間複雜度 Big O（先判斷再寫）

- $O(1)$ 常數 / $O(\log n)$ 對數（如二分搜尋） / $O(n)$ 線性 / $O(n \log n)$ （排序） / $O(n^2)$ 巢狀迴圈 / $O(2^n)$ 指數（未優化的遞迴）；下筆前先想清楚資料量級，避免用暴力解導致超時。

質數判斷

- 只需檢查到 $\sqrt{n}$ 即可判斷是否為質數：若 n 有大於 $\sqrt{n}$ 的因數，必定同時存在一個小於 $\sqrt{n}$ 的對應因數，所以檢查到 $\sqrt{n}$ 就足夠，不需要檢查到 n 。

階乘／費氏數列

- 階乘：迴圈遞乘 或 遞迴（ $n! = n \times (n-1)!$ ，注意 $n=0$ 時基準情況為 1）。費氏數列：遞迴寫法直觀但因「重複計算相同子問題」效率差（ $O(2^n)$ ），迭代寫法用兩個變數滾動更新效率佳（ $O(n)$ ），詳見下方圖解。

陣列常見操作

- 排序（內建 sort 即可，不需手刻）、去重（用 Set）、統計次數（用 Dictionary/Hash Map 累加），這三招可以應付大多數陣列題。

字串處理

- 反轉字串（切片或雙指標）、迴文判斷（從頭尾往中間比對是否相同）、計算字元出現次數與找出重複字元（用 Dictionary/HashMap，詳見下方圖解）。

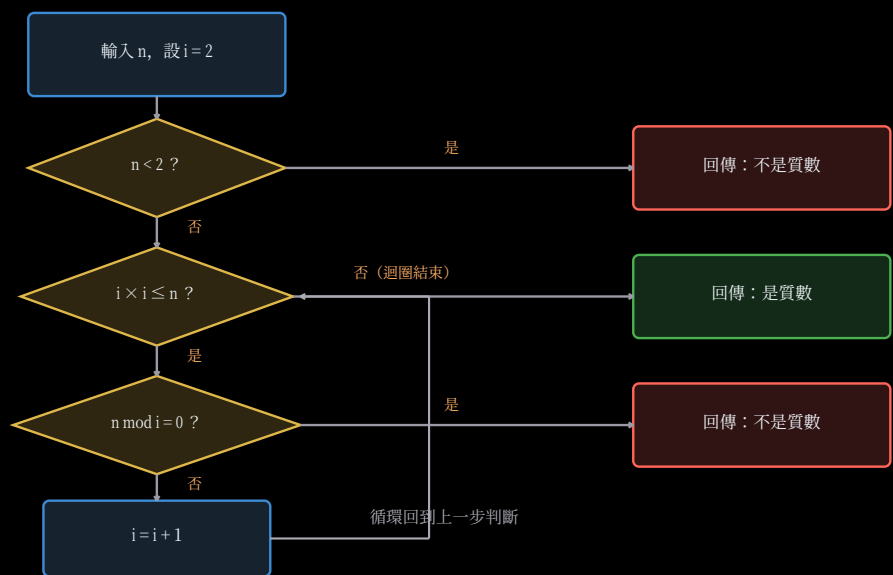
GCD/LCM（輾轉相除法）

- $GCD(a,b)$ ：若 $b=0$ 回傳 a ，否則遞迴呼叫 $GCD(b, a \bmod b)$ 。 $LCM(a,b) = a \times b \div GCD(a,b)$ 。

下筆前的檢查習慣

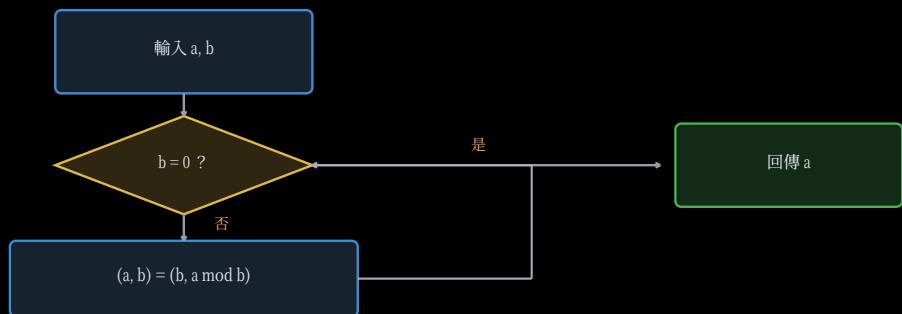
- 先想邊界條件（ $n=0$ 、負數、空輸入），再想時間複雜度，最後才寫程式碼——比直接下筆更不容易翻車。

圖解一：質數判斷 — 流程圖



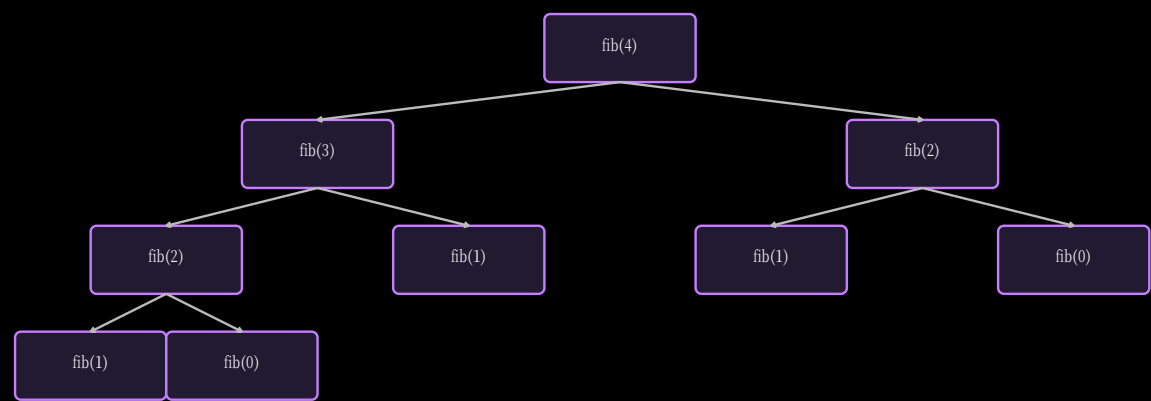
重點：迴圈只跑到 $i \times i \leq n$ (等同 $i \leq \sqrt{n}$)，不是跑到 n ，這是效率關鍵。

圖解二：GCD 輾轉相除法 — 流程圖



重點：每一輪把 (a, b) 換成 $(b, a \bmod b)$ ，數字會越來越小，最終 b 必為 0，時間複雜度 $O(\log(\min(a, b)))$ 。

圖解三：費氏數列 — 遞迴為何慢（以 fib(4) 展開）



fib(2) 被重複計算 2 次、fib(1) 被重複計算 3 次 —— 越層越多越喊張

作法	時間複雜度	空間複雜度	原理說明
原始遞迴	$O(2^n)$	$O(n)$ （呼叫堆疊深度）	如上圖，大量重複計算相同子問題， n 稍大（如 35 以上）就會明顯變慢
記憶化遞迴 Memoization	$O(n)$	$O(n)$	用字典快取算過的結果，遞迴前先查表，查到就直接回傳不再往下展開
迭代（滾動變數）	$O(n)$	$O(1)$	只保留前兩項數值逐步往前推進，不需要遞迴呼叫堆疊，最省空間

圖解四：字串內找重複字元 — HashMap 計數逐步走過

原理：用一個 HashMap（Dictionary）邊掃描字串邊累加次數，每個字元只需查詢/更新一次，整體時間複雜度 $O(n)$ 、空間複雜度 $O(k)$ （ k 為不同字元數量），比雙層迴圈兩兩比較的 $O(n^2)$ 解法快很多。以字串「banana」為例：

掃描順序	目前字元	HashMap 即時狀態
1	b	{b:1}
2	a	{b:1, a:1}
3	n	{b:1, a:1, n:1}
4	a	{b:1, a:2, n:1}
5	n	{b:1, a:2, n:2}
6	a	{b:1, a:3, n:2}

最終結果：a 出現 3 次、n 出現 2 次、b 出現 1 次 → 重複字元為 a、n。

圖解五：0/1 背包問題（Dynamic Programming） — DP 表格建構

原理：狀態轉移式 $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-weight[i]] + value[i])$ 。意思是對第 i 個物品，要嘛「不拿」（沿用前一列同容量下的最佳值），要嘛「拿」（騰出 $weight[i]$ 的容量，加上該物品價值），兩者取較大值。以 3 件物品（重量/價值分別為 1/15、3/20、4/30）、背包容量 4 為例：

物品數 i \ 容量 w	$w=0$	$w=1$	$w=2$	$w=3$	$w=4$
0 件（基準）	0	0	0	0	0
1 件（ $w_1=1, v_1=15$ ）	0	15	15	15	15
2 件（ $+w_2=3, v_2=20$ ）	0	15	15	20	35
3 件（ $+w_3=4, v_3=30$ ）	0	15	15	20	35

最終 $dp[3][4]=35$ （拿物品1+物品2：重量1+3=4、價值15+20=35，優於只拿物品3的 30）。

演算法評估總表（含你在校學過的進階題型）

演算法	核心原理（一句話）	時間複雜度	這次測驗可能性
質數判斷	試除法檢查到 $\sqrt{n}$	$O(\sqrt{n})$	高一—基礎數學題典型
GCD/LCM	輾轉相除法	$O(\log n)$	高一—基礎數學題典型
費氏數列/階乘	遞迴／記憶化／迭代	$O(n) \sim O(2^n)$	高一—Math Challenge 常客
字串找重複字元	HashMap 計數	$O(n)$	高一—你已學過，準備投報率最高
迴文判斷/字串反轉	雙指標由外往內比對	$O(n)$	中高
雙指標 Two Pointers	兩指標依條件相向或同向移動	$O(n)$	中
二分搜尋 Binary Search	每次排除一半搜尋範圍	$O(\log n)$	中——前提是資料已排序
排序原理（Quick/Merge）	理解即可，內建 sort 就好，不必手刻	$O(n \log n)$	低（手刻）／中（觀念）
0/1 背包問題 DP	狀態轉移式，見上方表格	$O(n \times \text{容量})$	中低——可能出簡化版
矩陣鏈乘法最佳化	DP 枚舉切割點，求最少乘法次數	$O(n^3)$	低——實作複雜，時限內少見
旅行商問題 TSP	NP-Hard，Held-Karp 狀態壓縮 DP 或近似解	$O(n^2 \times 2^n)$ 精確解	低——多為概念提問而非完整實作

延伸原理（你在校學過，機率偏低但建議能口述講清楚）

矩陣鏈乘法最佳化

- 矩陣乘法滿足結合律， $A \times B \times C$

的「結果」不變，但「先算哪一對」會讓中間需要的乘法運算次數天差地遠。用 DP 求解： $dp[i][j]$ 代表第 i 到第 j 個矩陣相乘的最小運算量，枚舉每個可能的切割點 k ，把問題拆成 $dp[i][k] + dp[k+1][j]$ + 切割點本身相乘的次數，取所有切法中的最小值。

旅行商問題 TSP

- 從一個起點出發，走訪所有城市各一次後回到起點，求總距離最短的路徑。暴力解需嘗試所有排列組合，時間複雜度 $O(n!)$ ，屬於 NP-Hard 問題（目前沒有已知的多項式時間演算法能保證找到最佳解）。Held-Karp 動態規劃用「位元遮罩 bitmask」表示「目前已拜訪過哪些城市的集合」當狀態，把複雜度降到 $O(n^2 \times 2^n)$ ，但 n 稍大（如大於 20）仍不可行，實務上會改用貪婪法或啟發式演算法（如遺傳演算法）求近似解。

為什麼機率偏低：這兩題屬於演算法課程中經典的動態規劃題型，概念完整但實作複雜，在 2.5 小時、雙題皆命名為「Math Challenge」的線上篩選測驗中，直接考「完整實作」的機率偏低，時間成本高、與職缺日常工作關聯性也較低；但仍可能被面試官延伸提問「這是什麼類型的問題、你會怎麼解」，建議把「原理講得清楚」列為優先，而非考前臨時手刻完整程式。

選擇/是非題	簡答/開放題	程式實作題
不適用（此類為程式實作題）。	TSP／背包問題／矩陣鏈乘法若被提及，多半是概念性提問（屬於什麼問題類型、如何求解），非要求完整實作。	考前務必手寫練習 2-3 題數學邏輯題（質數/費氏/GCD/找重複字元），確保完全不依賴查詢就能獨立寫出，因為系統會偵測切換分頁，考試中查詢空間有限。

九 後端安全基礎

常見追問題，展現「工程嚴謹度」的加分項

【原理】SQL Injection 之所以能得逞，根本原因是「把使用者輸入的資料，直接當成程式指令的一部分去拼接執行」——資料庫引擎無法區分這段文字究竟是「應該被當作資料處理的內容」還是「應該被當作指令執行的語法」，攻擊者只要讓輸入內容包含合法的 SQL

語法片段，就能讓資料庫誤把它當指令執行。參數化查詢（Prepared Statement）之所以能防範，是因為它把「SQL 語法結構」與「參數的實際值」分成兩個獨立管道分別送給資料庫引擎：語法結構先被編譯固定下來，之後不管參數值放什麼內容，都只會被當成「純資料」代入，不會被重新解析成語法，從機制上杜絕了注入的可能性。密碼雜湊同理：雜湊函式是「單向」的數學函式，從輸出反推輸入在計算上不可行，加鹽（salt，每個使用者加一段隨機字串再雜湊）則是為了讓相同密碼的使用者也產生不同的雜湊結果，防止攻擊者用預先算好的「雜湊值對照表 rainbow table」直接查出密碼。

SQL Injection 與防範

- 攻擊者把惡意 SQL 語法拼進輸入欄位，若後端直接字串拼接 SQL 就會被利用；防範方式是使用「參數化查詢/Prepared Statement」，讓輸入值永遠被當作資料而非可執行語法。

XSS（跨站腳本攻擊）

- 攻擊者把惡意 script 注入到網頁內容中，其他使用者瀏覽時被執行；防範方式是對輸出到頁面的內容做逸出處理（escape）。

密碼儲存原則

- 密碼絕不可明文儲存，需經過不可逆的雜湊演算法（如 bcrypt）加鹽（salt）處理，即使資料庫外洩也無法直接反推原始密碼。

JWT 使用重點

- 簽章需驗證（避免被竄改）、有效期不宜過長、不應把敏感資料明文放進 payload（JWT 內容僅編碼未加密，可被直接解讀）。

生活比喻

SQL Injection 就像「在填寫單據時夾帶暗語指令給承辦人員」——正常應該只是填寫姓名欄位，但如果承辦人員（資料庫）分不清「這是姓名」還是「這是要我執行的指令」，惡意填寫者就能讓承辦人員做出超出預期的事。參數化查詢就像先把單據格式印死、姓名欄位只能是文字，不管你填什麼都只會被當成姓名處理，不會被誤認成指令。密碼加鹽雜湊就像每個人的保險箱密碼都先混入專屬的隨機密語再上鎖，就算兩人設一樣的密碼，鎖起來的樣子也完全不同。

實務小例子（面試可直接這樣講）

「MineMarket 串接綠界金流時，我有做 CheckMacValue 簽章驗證，確保收到的付款通知真的是綠界發出、沒被竄改過的，這跟 JWT 簽章驗證是同一個安全設計精神：不只看內容，還要驗證來源與完整性有沒有被動過手腳。」

選擇/是非題	簡答/開放題	程式實作題
可能考 SQL Injection/XSS 基本定義判斷。	常見追問「你怎麼確保系統安全性」，可結合你 MineMarket 專案的 ECPay CheckMacValue 簽章驗證經驗回答。	較少直接考code。

十 機器學習與深度學習基礎

對應你正在學習的 DL/LLM/NLP，AI工程師職缺的知識底盤

【原理】機器學習的本質是「用資料自動找出一個函數 f ，讓輸入 x 對應到正確的輸出 y 」，而不是像傳統程式一樣把規則寫死。訓練的原理是：先定義一個「損失函數 Loss Function」衡量模型目前的預測值與真實答案差多少，再用最佳化演算法（最常見是梯度下降 Gradient Descent）反覆調整模型參數，讓損失越來越小，直到收斂。神經網路能「自動學習」的數學核心是反向傳播 Backpropagation：先讓資料往前跑過整個網路算出預測值與損失（前向傳播 Forward Propagation），再用微積分的連鎖法則 Chain Rule，從輸出端往回逐層算出「每一個參數對最終損失的影響程度（梯度）」，接著用梯度下降沿梯度反方向微調每個參數，這一來一回反覆數千數萬次，模型就逐漸學會任務。LLM 屬於深度學習底下的一支，差異在於用 Transformer 架構的自注意力機制處理「序列（文字）」資料，並在海量文字上做自監督式的預訓練（不需人工標註答案，用「預測下一個字」本身當作訓練目標）。

A. 傳統機器學習 Machine Learning

三大學習典範

- 監督式學習 Supervised Learning（有標準答案可對照，如分類/回歸）／非監督式學習 Unsupervised Learning（沒有標準答案，找資料內在結構，如分群）／強化學習 Reinforcement Learning（透過獎勵/懲罰訊號讓 Agent 學習策略）。

Regression vs Classification

- Regression（回歸）：預測連續數值（如房價）。Classification（分類）：預測離散類別（如垃圾郵件/非垃圾郵件）。

Overfitting／Underfitting（精準定義）

- Overfitting（過擬合）：模型把訓練資料的細節甚至雜訊都背下來，導致訓練表現很好、但遇到新資料表現差；緩解方式：正則化 Regularization（懲罰模型複雜度）、增加資料量、Early Stopping（提早停止訓練）。Underfitting（欠擬合）：模型太簡單，連訓練資料本身都學不好。

Train/Test Split 與 Cross Validation

- 資料切分成訓練集（用來學習）、驗證集（用來調整超參數）、測試集（最終評估，訓練過程不可看到）；K-fold 交叉驗證：把資料切成 K 份，輪流拿其中一份當驗證集、其餘 K-1 份訓練，跑 K 次取平均，讓評估結果更穩定、不受單一次切分運氣影響。

常見評估指標（混淆矩陣）

- 以二元分類為例：TP（True Positive，預測對且是正例）、FP（False Positive，預測是正例但實際不是）、FN（False Negative，預測不是但實際是）、TN（True Negative，預測對且是負例）。Precision（精確率）= $TP/(TP+FP)$ ，重視「別誤判」；Recall（召回率）= $TP/(TP+FN)$ ，重視「別漏抓」；F1-score 是兩者的調和平均，兼顧兩邊。

常見傳統演算法一句話原理

- Linear Regression：找一條線使誤差平方和最小。Logistic Regression：用 Sigmoid 函式把線性輸出壓到 0~1 之間做機率式分類。Decision Tree（決策樹）：依特徵不斷切分規則建樹。Random Forest（隨機森林）：訓練多棵決策樹投票取多數，靠集成學習 Ensemble Learning 降低單一樹的過擬合風險。KNN（K 最近鄰）：找最近的 K 個鄰居投票決定類別。K-means：非監督分群演算法，反覆更新群心位置直到收斂。SVM（支援向量機）：找一個能最大化「間隔」的分隔超平面。

B. 深度學習 Deep Learning

神經元與激活函數

- 神經元 Neuron：把輸入加權求和後，經過一個「激活函數 Activation Function」產生輸出。常見激活函數：ReLU（負值直接歸零、正值不變，計算簡單且能緩解梯度消失問題）／Sigmoid（壓到

0~1，常用於二元分類輸出）／Softmax（把多個數值轉成總和為 1 的機率分布，用於多分類輸出）。

Forward／Backward Propagation

- 前向傳播：資料依序通過每一層算出最終預測值與損失。反向傳播：用鏈鎖法則從輸出端往回逐層計算每個參數的梯度，是神經網路「自動學習」的數學核心。

Gradient Descent 與 Learning Rate

- 沿著損失函數的梯度反方向微調參數，逐步逼近最小損失；Learning Rate（學習率）決定每一步調整的幅度，設太大會震盪不收斂、設太小會收斂太慢。

CNN（卷積神經網路）

- 用「卷積核 filter」在圖片上滑動提取局部特徵（如邊緣、紋理），且同一個卷積核在整張圖共用參數，大幅減少參數量，擅長影像處理。

RNN／LSTM

- RNN（循環神經網路）：處理序列資料時，把「前一步的隱藏狀態」也當作輸入之一，讓網路具備類似記憶的能力。LSTM 透過門控機制（遺忘門/輸入門/輸出門）決定要保留、寫入或輸出多少資訊，緩解 RNN 在長序列上的梯度消失問題；如今在 LLM 領域已多半被 Transformer 取代。

C. LLM／NLP（直接對應你的 AI Agent 專案經驗，職缺核心）

Tokenization（斷詞/分詞）

- 把文字切成模型能處理的最小單位 Token（可能是字、詞或子詞 subword），模型實際上是在 Token 的世界裡運算，而非直接理解人類文字本身。

Embedding（詞嵌入）

- 把每個 Token 轉換成一組高維度向量，語意相近的詞在向量空間中的距離也相近，是讓神經網路能「理解」文字語意的關鍵前處理步驟。

Transformer／Self-Attention（自注意力機制）

- 現代 LLM 的核心架構。處理某個 Token 時，模型會同時計算它與句子中「所有其他 Token」的關聯權重（注意力分數），藉此捕捉長距離的語意關係，取代了 RNN 必須依序處理、難以捕捉長距離依賴的缺點。

Pretraining vs Fine-tuning

- Pretraining（預訓練）：在海量文字上學習語言的通用規律，成本極高、通常由模型廠商完成。Fine-tuning（微調）：在少量特定任務資料上，進一步調整已預訓練好的模型，讓它更貼合特定需求。

RAG（Retrieval-Augmented Generation，檢索增強生成）

- 讓 LLM 在回答前，先從外部知識庫檢索相關資料，再把檢索結果一併放入 Prompt 讓模型參考後回答；藉此補足模型「知識截止日期」與「不知道的內容」的限制，且比重新訓練/微調模型成本低很多。

Hallucination（幻覺）

- 模型在缺乏足夠依據時，仍會「自信地」生成看似合理但實際錯誤或捏造的內容；這也是為什麼你的 Threat Intel Agent 要求「強制結構化輸出＋完整記錄決策軌跡」來提高可信度與可追溯性的原因。

需求情境	適合方法	原理一句話
需要改變模型的「行為風格」（如固定用某種語氣/格式回答）	Fine-tuning（微調）	用特定任務資料調整模型參數本身
需要補充「即時或私有」知識（模型本來不知道的資料）	RAG（檢索增強生成）	回答前先查外部資料庫，不改動模型參數
需要快速調整「單次任務」的指令與輸出格式	Prompt Engineering	只改輸入的指令設計，成本最低、最快

生活比喻

機器學習訓練過程像「新手練習射飛鏢」：每丟一次（預測），看偏離靶心多遠（Loss，損失），再依偏離方向微調下一次的姿勢（梯度下降更新參數），丟得夠多次後越丟越準。Overfitting 像「死背考古題卻不懂原理的學生」，模擬考（訓練資料）分數超高，換一份新考卷（測試資料）就完全不會。Transformer 的注意力機制像「開會發言時，每個人講話都會同時留意在場所有人的反應與發言重點，而不是只顧自己講、不管別人」，藉此判斷這句話接下去怎麼講最合理。

實務小例子（面試可直接這樣講）

「我 Threat Intel Agent 裡強制模型第二輪一定要呼叫 submit_report 函式，就是為了避免 Hallucination（幻覺）——與其讓模型自由發揮生成一段看似專業但可能亂掰的文字，不如透過工程手法逼它一定照著我定義好的欄位結構回報，降低不可控風險，這也是我對『AI 工程』而非單純『用 AI』的理解。」

選擇/是非題	簡答/開放題	程式實作題
可能考 Overfitting／Precision／Recall／Token／Embedding 等名詞定義配對。	高機率——104 訊息明確會評估 AI 工具運用能力，且你履歷主打 AI Agent 開發，建議直接用 Threat Intel Agent 案例回答 Transformer/Tool Use/RAG/Hallucination 等概念題。	低機率——Coderbyte 的 Math Challenge 型測驗不太會考手刻機器學習演算法（如手刻梯度下降），除非該職缺額外客製化考題。

十一 Docker 容器化

解決「我電腦可以跑、你那邊卻不能跑」的環境問題

【原理】Docker 解決的是「環境不一致」問題：把應用程式連同它需要的作業系統函式庫、執行環境（如 Python 版本、套件版本）一起打包成一個「映像檔 Image」，這個映像檔在任何裝有 Docker 的機器上，都會產生完全一致的執行環境。技術上是靠 Linux 核心提供的 Namespace（命名空間，讓容器只看得到自己的檔案系統、網路、程序清單，看不到宿主機或其他容器的內容）與 Cgroups（限制容器能用多少 CPU、記憶體等資源上限）這兩項作業系統層級的隔離機制達成的「輕量虛擬化」。這與傳統虛擬機（VM）不同：VM 需要模擬整台電腦的硬體、在裡面裝一套完整的作業系統，啟動慢、資源消耗大；容器則是與宿主機共用同一個作業系統核心，只隔離「視角」而非整個系統，因此容器啟動快（秒級）、資源消耗低，能在同一台機器上同時跑更多個容器。

Image vs Container

- Image（映像檔）是唯讀的「模板/藍圖」，本身不會執行；Container（容器）是 Image 被啟動後、實際運行中的「實體」。一個 Image 可以同時開很多個 Container 實體，就像同一份設計圖能蓋出好幾間一模一樣的房子。

Dockerfile

- 描述「如何一步步建構出這個 Image」的文字檔：FROM 指定基底映像（如 python:3.11）、COPY 把程式碼複製進去、RUN 執行安裝指令（如 pip install）、CMD 定義容器啟動時要執行的指令。

Volume（資料卷）

- Container 本身預設是無狀態的，一旦容器被刪除或重啟，容器內寫入的資料會一併消失；Volume 是把外部持久化儲存空間「掛載」進容器的機制，讓資料能跨越容器的生命週期保留下來（這與你履歷提到 Heroku 檔案系統無狀態、需要用 Cloudinary 解決圖片遺失問題，是同一類問題的不同解法）。

Docker Compose

- 用一個 YAML 設定檔，定義「多個相關聯的容器」該如何一起啟動與串接（如後端 API 容器 + 資料庫容器 + Redis 容器），一個指令就能把整套服務環境架起來，不必一個一個手動啟動。

你的實戰對應

- Threat Intel Agent 「以 Docker 容器化並部署至雲端平台」，代表你已經有把應用打包成不依賴特定機器環境、隨處可重現的實務經驗。

生活比喻

Docker 映像檔就像「標準化貨櫃」：不管貨櫃裡裝的是什麼、原本要用什麼方式搬運，只要是標準貨櫃尺寸，任何貨櫃船、任何港口的吊車都能直接處理，不需要為每種貨物客製化搬運方式；Dockerfile 就是「打包貨櫃的清單/說明書」，寫清楚要放哪些東西、怎麼疊放。VM 則像是「連整艘船都要重新造一艘一樣的」，笨重很多。

實務小例子（面試可直接這樣講）

「假設面試被問為什麼要用 Docker，我會說：在 Threat Intel Agent 這個專案裡，我用到 MongoDB 跟好幾個外部 API 的環境變數設定，如果換一台機器重新架設，光是裝好 Python 版本、套件版本一致就很花時間，還容易漏東西；用 Docker 把整個執行環境寫進 Dockerfile 後，不管在我自己的電腦、還是部署到雲端平台，都是完全一樣的環境，一個指令就能重現，這對部署穩定性跟未來團隊協作都有幫助。」

選擇/是非題	簡答/開放題	程式實作題
可能考 Image vs Container、Docker vs VM（虛擬機）差異的判斷。	可能被問「為什麼你的專案要用 Docker 部署」，直接套用上方實務例子回答。	較少直接考 Dockerfile 撰寫，但操作手冊/履歷都提到 Docker/Render 部署經驗，可能被追問細節，建議能講清楚你實際的 Dockerfile 內容。

十二 CI/CD 與 DevOps 基礎

從程式碼推送到正式上線，如何自動化把關

【原理】CI/CD

解決的是「多人協作時，程式碼從寫完到上線這段過程容易出錯、又耗費人力」的問題。CI（Continuous Integration，持續整合）的原理是：每次有人把程式碼推送到共用分支，就自動觸發一連串檢查（如自動跑測試、程式碼風格檢查），盡早發現「我的改動跟別人的改動衝突/壞掉」，而不是等到最後要上線時才發現一堆問題疊在一起難以排查。CD（Continuous Delivery/Deployment，持續交付／部署）則是把「通過 CI 檢查的程式碼」自動（或半自動）部署到正式環境，減少人工手動部署容易出錯、忘記步驟的風險。整套 DevOps 文化的核心精神，是把「開發 Dev」與「維運 Ops」原本切割的兩個角色/流程，透過自動化工具串接起來，縮短「寫完程式碼」到「使用者真正用到」之間的時間與風險。

CI 持續整合

- 程式碼推送後自動觸發建構（Build）與測試（Test），常見工具：GitHub Actions、GitLab CI、Jenkins；目的是盡早（在合併進主分支「之前」）就發現問題，而不是事後才發現。

CD：Delivery vs Deployment 的精準區分

- Continuous Delivery（持續交付）：通過測試後自動打包成「可隨時部署」的版本，但正式上線這一步仍需人工按下確認按鈕。Continuous Deployment（持續部署）：通過測試後直接自動上線，全程無需人工確認，風險更高但速度最快。

Pipeline（流水線）

- 定義一連串自動化步驟的設定檔（如 `.github/workflows/xxx.yml`），描述「什麼事件觸發，依序跑哪些步驟，某個步驟失敗要怎麼處理（如中止並通知）」。

常見 Pipeline 步驟

- Lint（程式碼風格/語法檢查），Build（建構/打包，如 Docker image build），Test（自動化測試），Deploy（部署到 Staging 測試環境或 Production 正式環境），依序執行。

環境分層 Environment

- 通常區分 Development（開發環境）／Staging（預備/測試環境，盡量模擬正式環境）／Production（正式環境，真實使用者在使用），變更逐層驗證過關再往上推，降低直接影響正式環境的風險。

你可延伸的實戰經驗

- 履歷雖未直接寫 CI/CD 經驗，但你在 Railway／Heroku／Render 的部署經驗，本質上都屬於「CD」的範疇（程式碼推送後觸發部署）；自傳中「目前正在補強 Docker 容器化部署」這句話，也可以自然銜接到你正在補強 CI/CD 這個方向。

生活比喻

CI/CD 就像工廠的「品管流水線」：每個零件（每一次程式碼修改）一進到產線，就會自動經過一連串品管站（自動化測試），有問題馬上被攔下來退回，不會讓瑕疵品一路流到最後才被發現；通過品管的零件才會自動送進包裝出貨線（自動部署），不需要每次都靠人工搬運、人工檢查，也不會因為某個人手滑漏檢查一步而讓瑕疵品流出去。

實務小例子（面試可直接這樣講）

「假設我在 MineMarket 這個四人協作專案中要導入 CI，我會在 GitHub Actions 設定一個 Pipeline，每次有人推送 PR 時，自動跑我們寫的測試、檢查 Flask Blueprint 模組有沒有語法錯誤，測試沒過就不允許合併進主分支，這樣可以避免多人協作時，有人改了認證模組卻不小心影響到別人的購物車模組，卻沒被即時發現的狀況。」

選擇/是非題

簡答/開放題

程式實作題

可能考 CI vs CD 定義配對、常見工具名稱判斷。	可能被問「你了解 CI/CD 嗎／你的部署流程是怎樣」，可用上方實務例子延伸回答；若這塊經驗確實較薄弱，坦承「目前正在補強」也沒關係（呼應你自傳中已提到的方向），比硬掰更有說服力。	幾乎不會考實作，這是概念性理解題。
--------------------------------	--	-------------------